Machine Learning Assignment 2: Submission

M.Tech (AIML), BITS Pilani WILP | V Sujith Vishal | 2026aj05357

Submission Links

1. GitHub Repository Link

https://github.com/sujithvishal/2026AJ05357_ML_Assignment_2

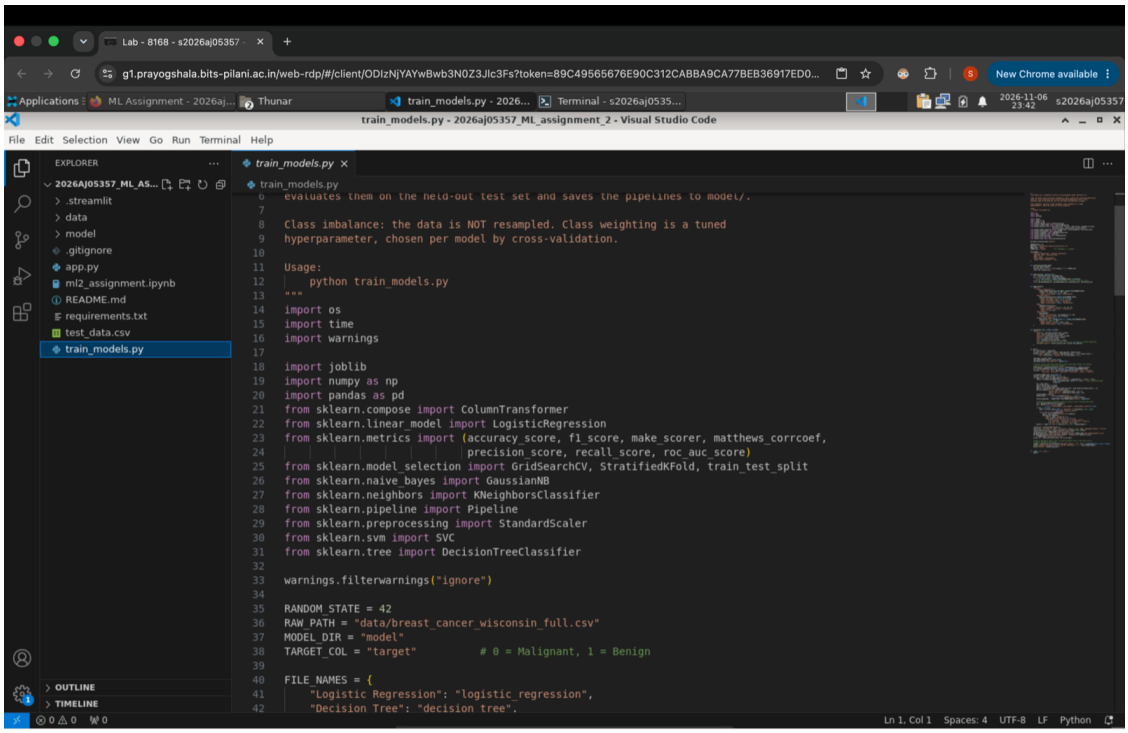
The repository contains the complete source code (`app.py`, `train_models.py`, `ml2_assignment.ipynb`), `requirements.txt`, `README.md`, the test data (`test_data.csv`) and the saved models for all five classifiers in `model/`.

2. Live Streamlit App Link

https://2026aj05357-ml-assignment-2.streamlit.app/

Deployed on Streamlit Community Cloud. The app opens an interactive frontend with CSV upload, model selection dropdown, evaluation metrics, and confusion matrix / classification report.

3. Screenshot: Assignment Execution on BITS Virtual Lab



Assignment executed on the BITS Virtual Lab

4. GitHub README Content

The README content follows (Problem statement, Dataset description, GitHub repository link, Models used with comparison table and observations).

Breast Cancer Classification — Model Comparison (ML Assignment 2)

Name: V Sujith Vishal **Course:** M.Tech AIML, BITS Pilani WILP **BITS ID:** 2026aj05357 **GitHub:** sujithvishal/2026AJ05357_ML_Assignment_2
Live app: 2026aj05357-ml-assignment-2.streamlit.app

> Note Class balance handling: the dataset was NOT resampled. - Data: no SMOTE, no over- or under-sampling. Train and test both keep the real ~63% Benign / ~37% Malignant ratio. - Training: class weighting (and 50/50 priors for Naive Bayes) is evaluated as a model-specific hyperparameter, and 5-fold CV decides per model whether it helps. KNN has no weighting option. - Tuning metric: F1 of the positive class (Benign = 1) inside cross-validation, not accuracy. - Final ranking: Malignant recall first (missing a malignant case is the costly error), then F1 and MCC as tie-breaks.

1. Problem Statement

This project classifies breast mass biopsies as **Malignant** (`target = 0`) or **Benign** (`target = 1`) from measurements of cell nuclei computed from digitized Fine Needle Aspirate (FNA) images. Accurate automated screening matters in healthcare: missing a malignant mass is costly, and the 30 measurements are strongly correlated, so a systematic comparison of different model families is useful. The goal of this assignment is to build, compare and evaluate five classification algorithms on this task, tune each one's hyperparameters with cross-validation, decide how to handle the (mild) class imbalance, identify which model generalizes best, and deploy the result as an interactive web app that scores new data without retraining.

2. Dataset Description

- **Name:** Breast Cancer Wisconsin (Diagnostic) Dataset
- **Source:** UCI Machine Learning Repository, https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic
- **Size:** 569 instances, 30 numeric features + 1 target (no missing values, no duplicates)
- **Used in this project:** all 569 rows, split 80/20 (stratified on the target, `random_state=42`) into train (455) and test (114)
- **Target:** `target` — `1` = Benign (357 rows, 62.7%), `0` = Malignant (212 rows, 37.3%)
- **Features (30):** ten nuclear properties — radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, fractal dimension — each given as a **mean**, a **standard error** (`*_error`) and a **worst** (largest) value (`worst_*`).
- **Preprocessing:** features are standardized (`StandardScaler`) inside every model pipeline, so scaling is refitted on each CV training fold with no leakage and each saved model accepts raw feature columns. No rows or columns are dropped.
- **Class imbalance:** 62.7% Benign vs 37.3% Malignant (about 1.7:1). The data is **not resampled** and the test set keeps the real-world ratio. SMOTE was rejected: the imbalance is mild, the training set is small (455 rows), and it would add `imbalanced-learn` as an app dependency.

This satisfies the assignment's requirements of ≥500 instances (569), ≥12 features (30) and a binary target sourced from UCI.

EDA highlights (full analysis in `ml2_assignment.ipynb`)

- Malignant masses have clearly larger nuclei; radius, perimeter, area and concave-point values are shifted well above the Benign distribution.
- Strongest signals: `worst_concave_points`, `worst_perimeter`, `mean_concave_points`, `worst_radius` (correlation with `target` of about −0.74 to −0.79).
- Strong multicollinearity: 21 feature pairs have |r| > 0.9 (e.g. `mean_radius` vs `mean_perimeter`, r = 0.998). This hurts Naive Bayes' independence assumption and makes KNN count the same information repeatedly.

3. GitHub Repository

GitHub repo link: https://github.com/sujithvishal/2026AJ05357_ML_Assignment_2
Live Streamlit app: https://2026aj05357-ml-assignment-2.streamlit.app/

The repository contains the complete source code (`app.py`, `train_models.py`, `ml2_assignment.ipynb`), `requirements.txt`, this README, the test data (`test_data.csv`) and the five saved models in `model/`.

4. Models Used and Comparison

Hyperparameter tuning

Every model is a `Pipeline(StandardScaler + classifier)` tuned with `GridSearchCV` using **5-fold stratified cross-validation on the 455-row training set only**. Each combination is scored on F1, MCC, AUC, Accuracy, Precision and Recall, and the best is refitted by **F1** (Benign as the positive class).

Model	Grid searched	Best (selected by CV F1)
Logistic Regression	C ∈ {0.01, 0.1, 1, 10, 100}, class_weight ∈ {None, balanced}	C=0.1, class_weight=None
Decision Tree	max_depth ∈ {3, 4, 5, 6, 8, None}, min_samples_leaf ∈ {1, 5, 10, 20}, criterion ∈ {gini, entropy}, class_weight	criterion='entropy', max_depth=4, min_samples_leaf=5, class_weight=None
KNN	n_neighbors ∈ {3, 5, 7, 11, 15, 21}, weights ∈ {uniform, distance}, p ∈ {1, 2}	n_neighbors=3, uniform, p=1
Naive Bayes (Gaussian)	var_smoothing ∈ 10^{-9} … 10^{0} (10 values), priors ∈ {natural, 50/50}	var_smoothing=0.1, priors=None
SVM (RBF)	C ∈ {0.1, 1, 10}, gamma ∈ {scale, 0.001, 0.01}, class_weight	C=10, gamma=0.01, balanced

Comparison table (114-row held-out test set)

All six metrics computed with `sklearn.metrics` on the untouched test split.

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC	Malignant recall
Logistic Regression	**0.9737**	0.9957	0.9726	**0.9861**	**0.9793**	0.9433	0.9524
Decision Tree	0.8772	0.9593	0.9531	0.8472	0.8971	0.7542	0.9286
KNN	0.9649	0.9714	0.9595	**0.9861**	0.9726	0.9245	0.9286
Naive Bayes	0.9474	0.9878	0.9459	0.9722	0.9589	0.8864	0.9048
Support Vector Machine	**0.9737**	**0.9967**	**0.9859**	0.9722	0.9790	**0.9439**	**0.9762**

(Bold = best value per metric.) Precision, Recall and F1 treat **Benign (1)** as the positive class, as in the assignment's table. **Malignant recall** is the recall of the malignant class (sensitivity): the share of malignant biopsies that are caught. It is the priority metric, because a missed malignant case is the costly error. With only 114 test rows (42 malignant), one patient is about 0.9 accuracy points and 2.4 points of Malignant recall, so small differences are within noise.

Malignant cases caught on the test set:

Model	Malignant caught	Malignant missed (called Benign)	Benign wrongly flagged Malignant
Support Vector Machine	41 / 42	1	2
Logistic Regression	40 / 42	2	1
KNN	39 / 42	3	1
Decision Tree	39 / 42	3	11
Naive Bayes	38 / 42	4	2

Should the dataset be balanced? (CV evidence, training data only)

Best configuration with vs without class weighting, 5-fold CV means:

Model	Weighting	CV Accuracy	CV AUC	CV Recall	CV F1	CV MCC
Logistic Regression	**natural**	0.9824	0.9951	0.9965	**0.9861**	**0.9627**
Logistic Regression	balanced	0.9758	0.9958	0.9754	0.9805	0.9490
Decision Tree	**natural**	0.9363	0.9566	0.9509	**0.9493**	**0.8654**
Decision Tree	balanced	0.9341	0.9514	0.9474	0.9472	0.8608
Naive Bayes	natural	0.9407	0.9878	0.9719	0.9535	0.8730
Naive Bayes	50/50 priors	0.9407	0.9878	0.9719	0.9535	0.8730

Model	Weighting	CV Accuracy	CV AUC	CV Recall	CV F1	CV MCC
SVM	natural	0.9758	0.9964	0.9895	0.9810	0.9486
SVM	**balanced**	0.9780	0.9962	0.9860	**0.9825**	**0.9533**

Important The dataset itself was not balanced. The rows stay at the natural ~63/37 ratio. At this mild imbalance weighting barely matters: CV kept the natural ratio for Logistic Regression, Decision Tree and Naive Bayes (identical scores), and chose balanced weights only for the SVM, by a margin of about 0.0015 F1. KNN has no weighting option.

Observations on model performance

Model	Tuned hyperparameters	Observation
Logistic Regression	C=0.1, class_weight=None	Best F1 (0.979), Benign recall 0.986 (tied with KNN), Malignant recall 0.952 (2 of 42 malignant missed), accuracy 0.974, MCC 0.943, AUC 0.996. The tuned C=0.1 means fairly strong regularisation, which suits the heavily collinear radius/perimeter/area features. The classes are close to linearly separable once scaled. Easiest to explain through its coefficients.
Decision Tree	criterion='entropy', max_depth=4, min_samples_leaf=5	Clearly the weakest: accuracy 0.877, recall 0.847, F1 0.897, MCC 0.754. Axis-aligned splits approximate the smooth, oblique boundary between correlated size features poorly, and a single tree has high variance on 455 training rows. CV kept it shallow to limit overfitting. Its strength is readable rules.
KNN	n_neighbors=3, weights='uniform', p=1	Accuracy 0.965 and Benign recall 0.986 (tied best), F1 0.973, but Malignant recall is only 0.929 (3 of 42 missed), below Logistic Regression and SVM. Manhattan distance beat Euclidean in 30 dimensions. Its AUC (0.971) is lower than the others because a 3-neighbour vote only produces four distinct probability values, which limits ranking quality. Needs scaling, which the pipeline provides.
Naive Bayes	var_smoothing=0.1, priors=None	Accuracy 0.947, F1 0.959, MCC 0.886, yet AUC 0.988 is much better than its accuracy suggests: it ranks cases well while its probabilities are poorly calibrated. The independence assumption is clearly violated by the near-duplicate size features, which costs it threshold-based accuracy.
Support Vector Machine	C=10, gamma=0.01, class_weight='balanced'	Best **Malignant recall (0.976, only 1 of 42 malignant missed)**, best AUC (0.997), precision (0.986) and MCC (0.944); F1 (0.979) is 0.0003 below Logistic Regression, accuracy identical. The RBF kernel with a small gamma gives a smooth, near-linear boundary.
Overall winner	**Support Vector Machine**	Ranked by Malignant recall first, then F1 and MCC. The SVM catches the most malignant cases (0.976, 1 of 42 missed), ahead of Logistic Regression (0.952, 2 missed), and also has the best AUC, precision and MCC; its F1 is 0.0003 below Logistic Regression. The gap to Logistic Regression is one patient, so the two are effectively tied on 114 test rows. Logistic Regression is the simpler, more explainable alternative.

Overall Ranking

Rank	Model	Malignant recall	F1	MCC	AUC	Accuracy	In short
1	**Support Vector Machine**	**0.976**	0.979	**0.944**	**0.997**	0.974	Catches the most malignant cases; best AUC, precision and MCC
2	Logistic Regression	0.952	**0.979**	0.943	0.996	0.974	Near-tie: best F1, Benign recall 0.986 (tied with KNN), simplest to explain
3	KNN	0.929	0.973	0.925	0.971	0.965	Benign recall 0.986, but misses 3 malignant cases; coarse probabilities
4	Decision Tree	0.929	0.897	0.754	0.959	0.877	Interpretable, but misses 3 malignant cases and wrongly flags 11 benign ones
5	Naive Bayes	0.905	0.959	0.886	0.988	0.947	Ranks well, but misses 4 malignant cases; independence assumption costs accuracy

Ranked by Malignant recall, then F1, then MCC. KNN and Decision Tree tie on Malignant recall; KNN's higher F1 places it above.

Winner: Support Vector Machine (Logistic Regression is a near-tied co-winner). Recall on the malignant class is the priority metric because a missed malignant case is the costly error; F1 and MCC break ties.

5. Repository Structure

```
.
├── app.py                    # Streamlit app (loads saved models, scores uploaded CSV)
```

```
├── train_models.py             # Standalone training script (same results as the
 notebook)
├── ml2_assignment.ipynb        # Notebook: EDA, imbalance analysis, tuning, eval
uation, saving
├── requirements.txt
├── README.md
├── test_data.csv               # Held-out 114-row test split (30 features + targ
et)
├── .streamlit/config.toml      # Custom app theme
├── .gitignore
├── model/
│   ├── logistic_regression.joblib
│   ├── decision_tree.joblib
│   ├── knn.joblib
│   ├── naive_bayes.joblib
│   ├── svm.joblib
│   ├── comparison_table.csv      # Test-set metrics for all 5 models
│   ├── best_hyperparameters.csv  # GridSearchCV winners (shown in the app)
│   └── class_balance_cv.csv      # CV metrics: balanced vs natural class weighti
ng
└── data/
    └── breast_cancer_wisconsin_full.csv   # Full dataset (569 rows)
```

6. Running Locally

```
python3 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt

python train_models.py           # optional: retrain and regenerate model/ and te
st_data.csv
streamlit run app.py
```

Upload `test_data.csv` (or use the sidebar's download button to get it), then pick a model from the dropdown. To run the notebook, also `pip install ipykernel jupyter`.

> **Version note:** the saved `.joblib` pipelines were created with `scikit-learn==1.6.1`, which is pinned in `requirements.txt`. Loading them with a different scikit-learn version can fail with an unpickling error. If you retrain with `train_models.py`, the models are regenerated for whichever version you have installed.

7. Streamlit App

The app never retrains: it loads the saved pipelines from `model/` and scores the uploaded CSV.

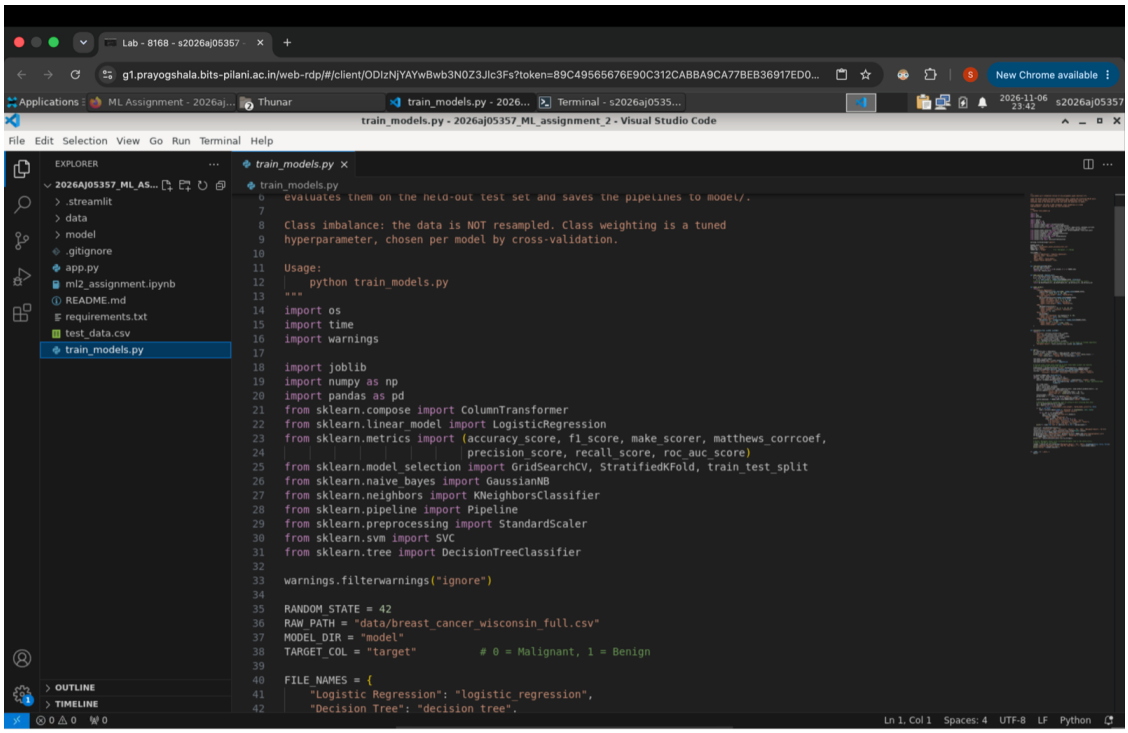
Required feature	How the app provides it
Dataset upload (CSV)	Sidebar "Upload my own CSV" option (or the bundled 114-row test sample / the full 569-row dataset), with column and label validation, plus a download button for test_data.csv
Model selection dropdown	**Model inspector** tab: dropdown with all 5 models; the tuned hyperparameters of the chosen model are shown
Evaluation metrics	Accuracy, AUC, Precision, Recall, F1 and MCC for the selected model, plus Malignant recall
Confusion matrix / classification report	Confusion matrix and ROC curve side by side, then the classification report
Results of all models	**Leaderboard** tab: ranked table, one bar panel per metric, combined ROC curves and a best-model banner (Malignant recall first, then F1 and MCC)

Live app: https://2026aj05357-ml-assignment-2.streamlit.app/

8. BITS Virtual Lab Execution

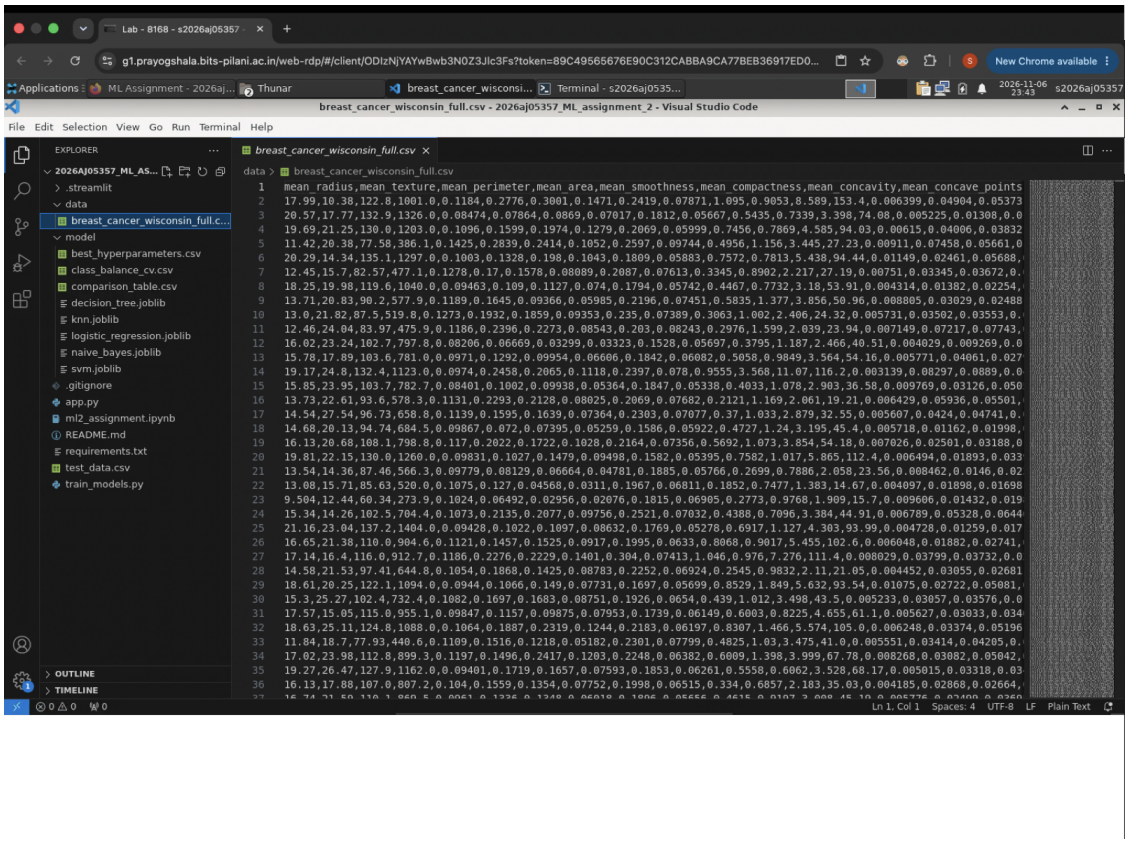
The assignment was executed on the BITS Virtual Lab; the screenshots are shown below and are also included in the submission PDF.

8.1 Lab environment: training script in VS Code



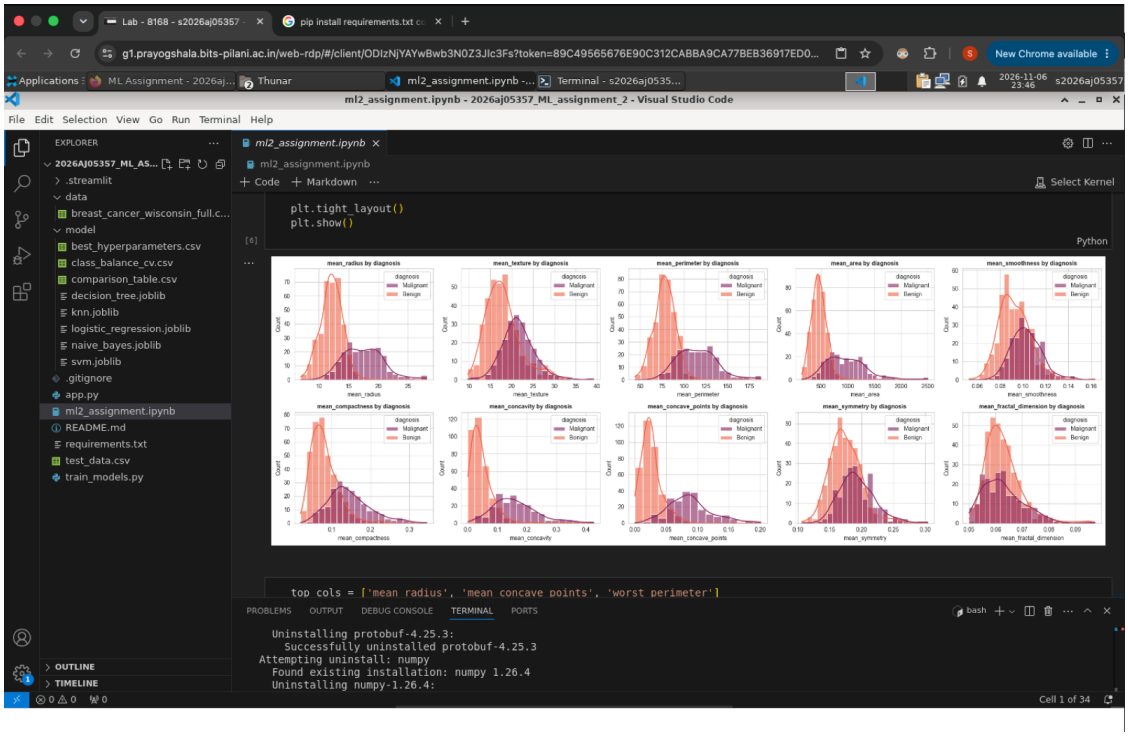
BITS Virtual Lab: train_models.py open in VS Code

8.2 Dataset: Breast Cancer Wisconsin CSV loaded in the lab



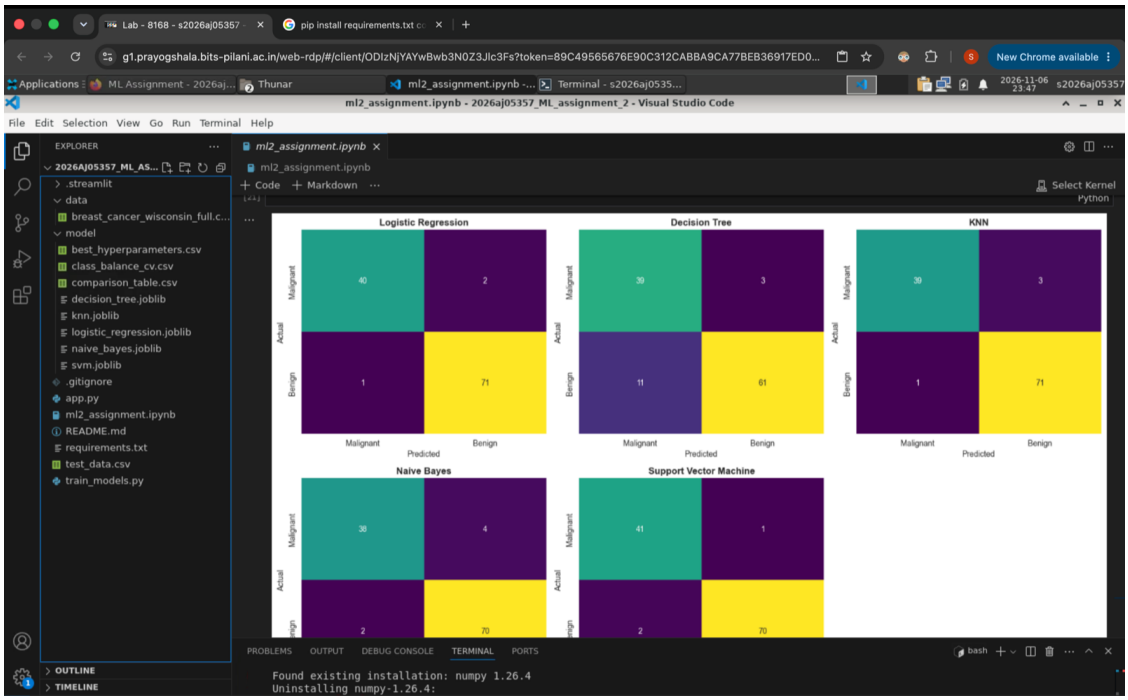
Dataset: breast_cancer_wisconsin_full.csv

8.3 Exploratory data analysis: feature distributions by diagnosis



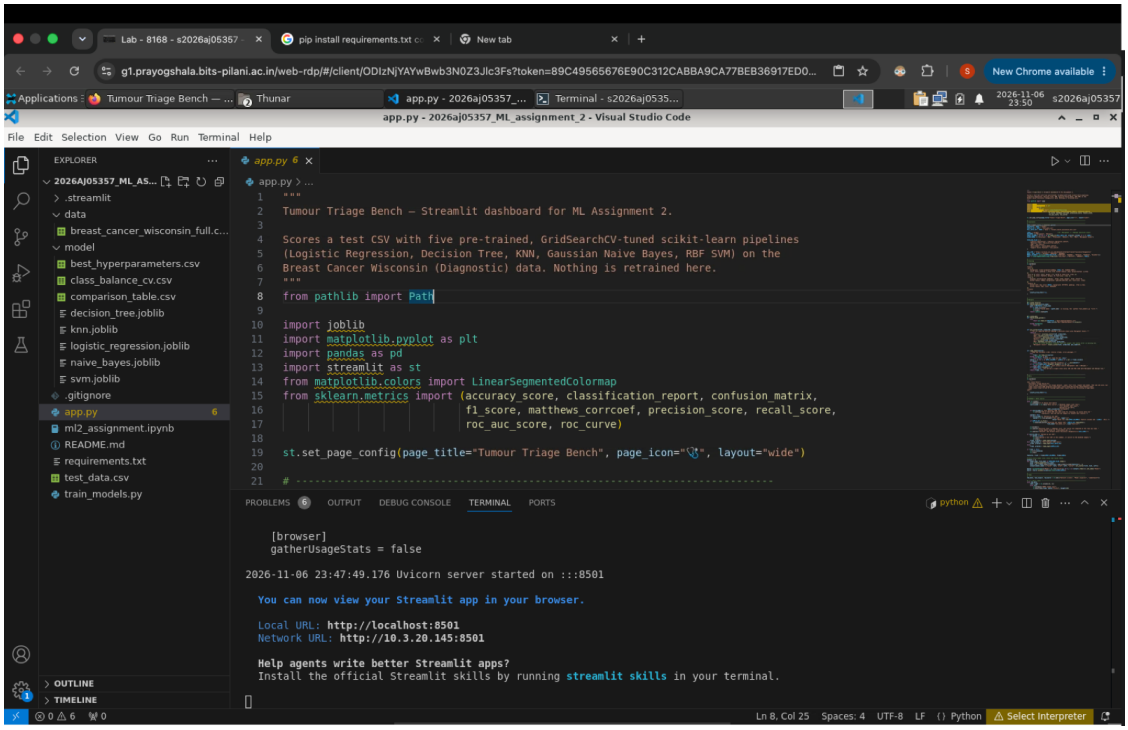
EDA: feature histograms by diagnosis

8.4 Confusion matrices of all five models (notebook output)



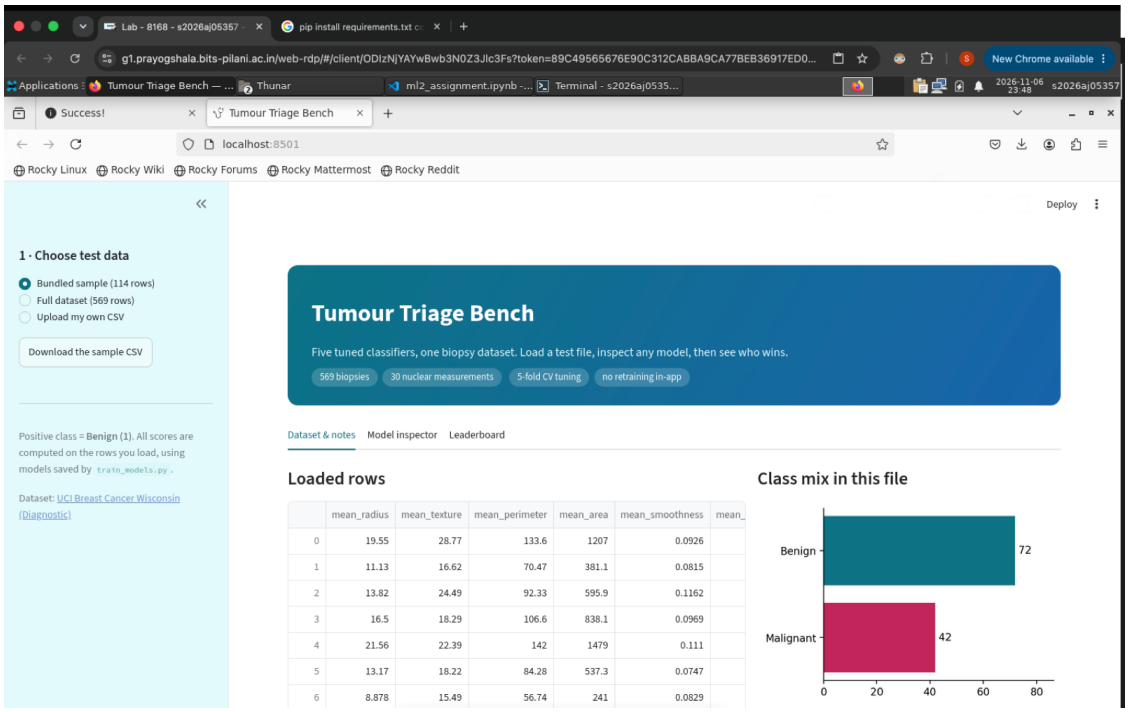
Confusion matrices of the five models

8.5 Streamlit app source and server start-up in the lab



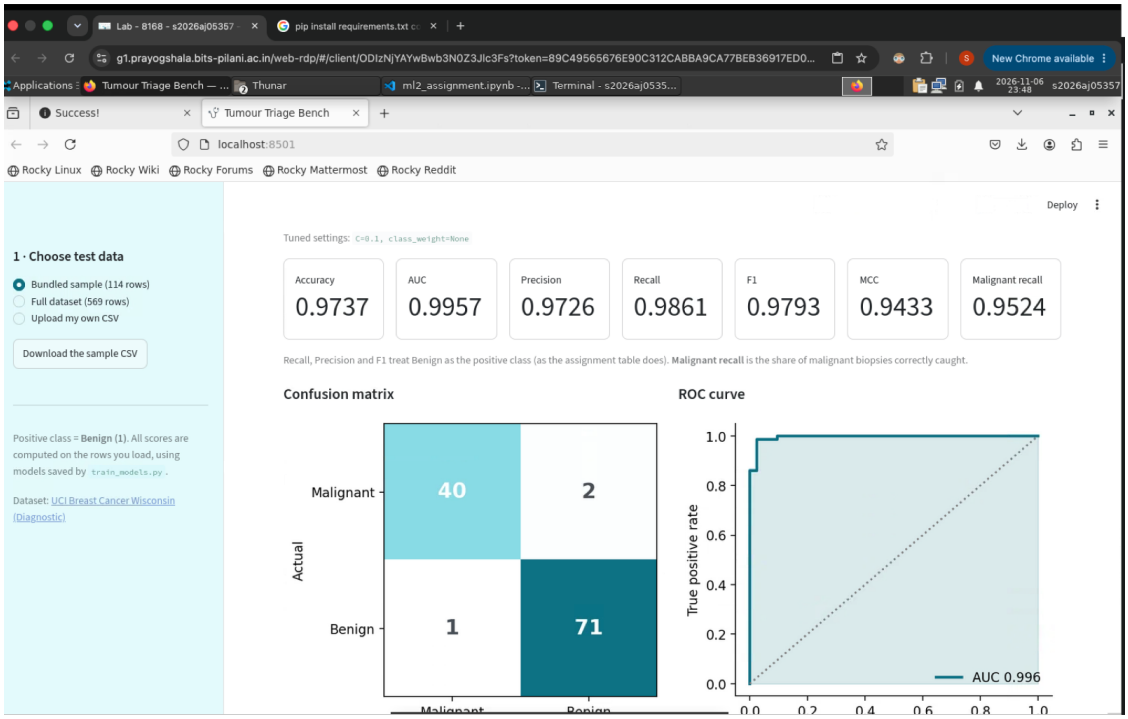
Streamlit app.py and server running on port 8501

8.6 Streamlit app: dataset and notes tab



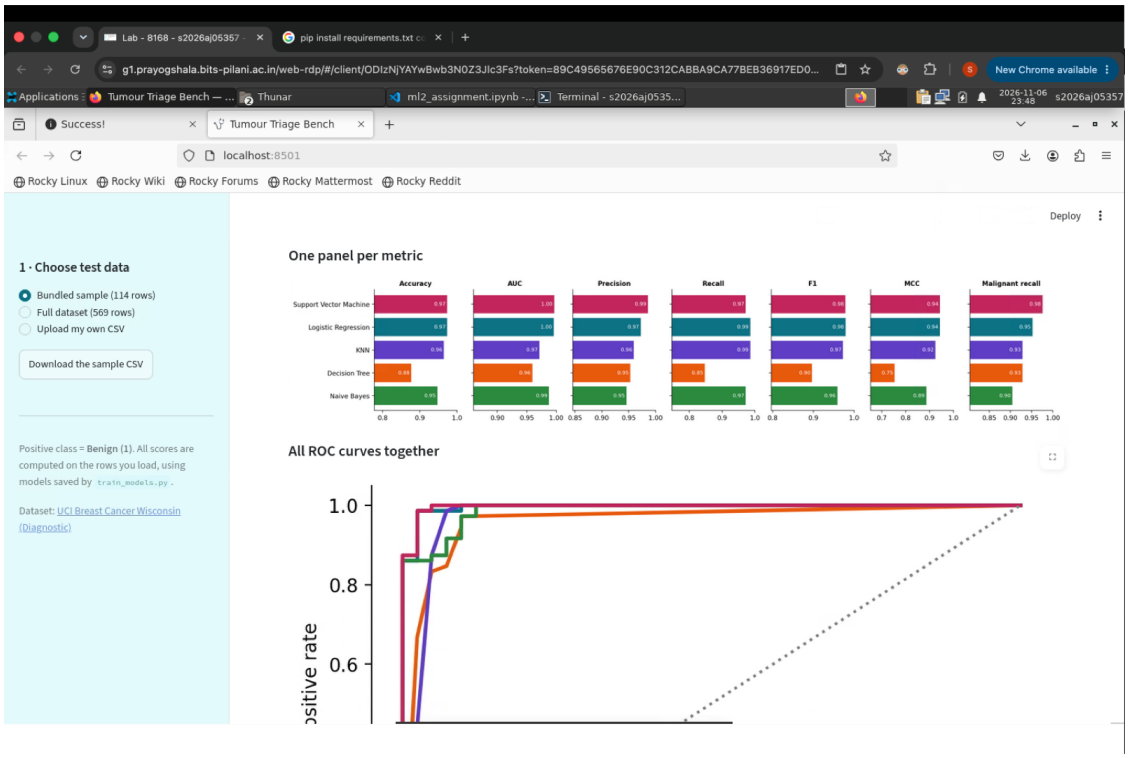
Tumour Triage Bench: dataset and class mix

8.7 Streamlit app: model inspector (metrics, confusion matrix and ROC curve)



Tumour Triage Bench: model metrics, confusion matrix and ROC curve

8.8 Streamlit app: leaderboard (metric comparison and combined ROC curves)



Tumour Triage Bench: leaderboard comparison graph